

Note: This exercise is to be done in **groups**. Submission is a .zip file.

For this exercise, using an Ubuntu Linux-based environment on a real x86_64 computer is necessary. (WSL Ubuntu will work, but Mac systems will not work as expected.)

Create a C++ program called **mytimer.cpp** that creates *two child processes* using **fork()**.

1. The parent shall loop indefinitely while printing out the current date/time after every 3 seconds.

- (2 pts) The date/time should have the format:
[yyyy-mm-dd] hh:mm:ss
You may use any time function in order to get the current date/time.
- (2 pts) Do NOT busy-wait for 3 seconds. (That is, don't just use a loop that checks the current time, with no other operation in the loop.)
 - There are several ways to "sleep" your process using Unix system calls, much like **Thread.sleep()** in Java – research about them!
- (2 pts) For every 3rd print out, print an extra message after printing the date/time:
"This program has gone on for far too long. Close the myXclock window or press Enter on this window to exit."
This extra message should include the quotation marks.

2. The first child process should use one of the **exec()** family functions to start the **xclock** program.

- (2 pts) Use the command **whereis xclock** or **which xclock** to find out where the program is, and use the reported path to start the program in your **exec()** call.
 - If you find that you don't have **xclock** in your system, you will need to manually install it. You can use the following command: **sudo apt install x11-apps**
- (2 pts) The process name should be set to **myXclock**. In other words, typing **ps aux** in a different terminal window (while your program is running) should reveal a process named **myXclock**.

3. The second child process should simply wait for the Enter key to be pressed.

- (1 pt) You can do this by simply asking for input using **getline(cin, ...)**.
- (1 pt) This child process should then print the message **"Terminated."** (with quotes) and the process must exit.

4. If you implemented all of the above, you may notice that closing the clock window or pressing Enter in the terminal does NOT actually end the parent process... yet. You should implement a way for the parent to wait on its children processes.

- (2 pts) If any one child finishes execution (or is killed), the parent itself should exit.
- (2 pts) If the parent exits, all of its children should be given a signal to exit as well.

- Hint: The parent process knows its children's PIDs when you did the **fork()**. What can you possibly do with those PIDs?...

(4 pts) Submit a zip file containing the following:

- `mytimer.cpp`
- accomplished COA: `Lab7_(surnames in alphabetical order)_COA.pdf`
- name the zip file: `Lab7_(surnames in alphabetical order).zip`

Example submission (the other files are contained within the zip file):

- `Lab7_EspinosaManaogSanchez.zip`
 - `mytimer.cpp`
 - `Lab7_EspinosaManaogSanchez_COA.pdf`

There will be no defense for this lab, but you are expected to produce fully-commented code.

As usual, if you did any research, declare your sources.